# PROJECT DOCUMENTATION

Implement Client Script & UI Policy (Incident)

| | |
|---|---|
| **Team ID** | SWTID-2026-1920 |
| **Team Leader** | Sandhiya Shree S |
| **Team Members** | Sayanthini T; Vindhiya V |
| **Date** | 29 October 2026 |
| **Platform** | ServiceNow |

## Project Summary

This project demonstrates how ServiceNow UI Policies and Client Scripts can guide user input, control field behaviour and prevent invalid updates on the Incident table. The configuration focuses on High impact incidents and improves the consistency of incident records when they are created or edited.

## 1. Introduction

ServiceNow is a cloud-based platform used to manage digital workflows, including IT service management. Incident records need consistent information so that support teams can triage, assign and resolve issues effectively. UI Policies and Client Scripts enforce form behaviour and validation rules at the user-interface level, before a record is submitted.

## 2. Problem Statement

When incident forms are completed manually, important fields may be left empty or set inconsistently. This affects reporting accuracy, SLA compliance and service quality. High impact incidents need stronger controls: assignment information must be provided, Urgency must align with Impact, and users should update records through the Incident form instead of editing State inline in the list.

## 3. Objectives

- Make **Assignment group** mandatory when Impact is High (UI Policy).
- Make **Urgency** read-only when Impact is High (UI Policy Action).
- Automatically set Urgency to High when Impact changes to High (onChange).
- Prevent submission of a High impact incident when **Assigned To** is empty (onSubmit).
- Block inline list editing of the **State** field (onCellEdit).
- Validate the configured behaviour using test cases.

## 4. Tools and Technologies

| Tool / Feature | Purpose |
|---|---|
| ServiceNow Incident table | Stores incident records and fields such as Impact, Urgency, Assignment group, Assigned To and State. |
| UI Policy and UI Policy Action | Controls field behaviour (mandatory, read-only, visible) based on a form condition. |
| Client Scripts | Run client-side logic on field change (onChange), form submission (onSubmit) and list-cell editing (onCellEdit). |
| JavaScript / g_form API | Implements form validation and field updates (getValue, setValue, addInfoMessage, showErrorBox). |

## 5. Implementation Details

Navigation: UI Policies are created from **System UI** → **UI Policies** and Client Scripts from **System UI** → **Client Scripts**, using an account with administrative or configuration access.

### 5.1 UI Policy: High Impact Control

| | |
|---|---|
| **Table** | Incident |
| **Name** | High Impact Control |
| **Active** | True |
| **Condition** | Impact is 1 - High |
| **Reverse if false** | True |

**UI Policy Actions**

| | |
|---|---|
| **Assignment group** | Mandatory = True |
| **Urgency** | Read-only = True; Visible = unchanged (field stays visible) |

The policy applies the field behaviour when Impact is High and reverses it automatically when the condition becomes false.

**Note:** the UI Policy makes **Assignment group** mandatory. The **Assigned To** check is done separately by the onSubmit script (5.3).

### 5.2 onChange Client Script: Impact

| | |
|---|---|
| **Name** | Auto set urgency for high impact |
| **Table** | Incident |
| **Type** | onChange |
| **Field name** | Impact |
| **Active** | True |

Sets Urgency to High when Impact changes to High. The loading check prevents the script from running while the form is loading.

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }

    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
    }
}
```

### 5.3 onSubmit Client Script: Assigned To validation

| | |
|---|---|
| **Name** | Prevent save if Assigned To missing |
| **Table** | Incident |
| **Type** | onSubmit |
| **Active** | True |

Stops submission if Impact is High and Assigned To is empty; otherwise it allows the save.

```
function onSubmit() {
    if (g_form.getValue('impact') == '1' &&
        g_form.getValue('assigned_to') == '') {

        g_form.showErrorBox(
            'assigned_to',
            'Assigned To is mandatory for High impact incidents.'
        );
        return false;
    }
    return true;
}
```

### 5.4 onCellEdit Client Script: State

| | |
|---|---|
| **Name** | Prevent state change via list edit |
| **Table** | Incident |
| **Type** | onCellEdit |
| **Field name** | State |
| **Active** | True |

Shows an alert and cancels inline list editing of State. Changes made on the Incident form are not affected.

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {

    alert('State cannot be updated using list editing. Please open the Incident.');

    callback(false);
}
```

## 6. Testing Plan

| No. | Test Action | Expected Result |
|---|---|---|
| 1 | Open a new Incident and set Impact to High. | Assignment group becomes mandatory; Urgency becomes read-only. |
| 2 | With Impact = High, check Urgency. | Urgency is set to High automatically by the onChange script and an info message is shown. |
| 3 | Leave Assigned To empty and click Submit. | Submission is blocked and the error 'Assigned To is mandatory for High impact incidents.' is shown. |
| 4 | Fill Assigned To (and Assignment group) and click Submit. | The Incident is saved successfully without errors. |
| 5 | Change Impact from High to Medium on an existing Incident. | Reverse-if-false applies: Assignment group is no longer mandatory and Urgency is editable; the record saves. |
| 6 | In the Incident list, double-click State to edit it. | An alert is shown, the list edit is cancelled and the State value stays unchanged. |
| 7 | Open the Incident form, change State and click Update. | The State change is saved successfully; the other rules still apply. |

Note: the results above are the expected outcomes of each test case.

## 7. Expected Results

The configuration guides users toward complete incident records, aligns Urgency with High impact, prevents submission when Assigned To is missing for High impact incidents, and blocks inline list editing of State.

## 8. Conclusion

This project applies ServiceNow UI Policies and Client Scripts to improve Incident form usability and data consistency. The UI Policy controls field behaviour based on Impact, while the Client Scripts respond to field changes, validate submission and restrict list editing. Together they demonstrate practical client-side controls in ServiceNow IT Service Management.